

Form3
Design
Title :

GROUP MEMBER:

No.	Student Name	Student ID
1.	Muhammad Hafid Al Asyrofi	
2.	Muhammad Kasyfil Aziz Setiyadi	
3.	Muhammad Habibi Dzikra	

Advisor : Sir. Nurhadi Sukmana

Submitted for
Capstone Design Project
to Faculty of Computer Science
President University

TABLE OF CONTENT

STATEMENT OF ORIGINALITY

In my capacity as an active student at President University and as the author of the Capstone Design Project stated below:

Name : 1. Muhammad Hafid Al Asyrofi – 001202300224

2. Muhammad Kasyfil Aziz Setiyadi – 001202300022

3. Muhammad Habibi Dzikra – 01220230004

Faculty : Computer Science

I hereby declare that my Capstone Design Project entitled "**Optimization Business Flow using Warehouse Management**" is to the best of my knowledge and belief, an original piece of work based on sound academic principles. If there is any plagiarism detected in this final project, I am willing to be personally responsible for the consequences of these acts of plagiarism and will accept the sanctions against these acts in accordance with the rules and policies of President University.

I also declare that this work, either in whole or in part, has not been submitted to another university to obtain a degree.

Cikarang, January 2024

Signer 1	Signer 2	Signer 3
Muhammad Hafid Al Asyrofi – 001202300224	Muhammad Kasyfil Aziz Setiyadi – 001202300022	Muhammad Habibi Dzikra – 01220230004

SCREENSHOT OF ZEROGPT

PART 3
DESIGN

Consists of:

- **SYSTEM DESIGN**
-
- **STANDARDS USED**
- **FUNCTIONAL TESTING**

A. SYSTEM DESIGN

1. High-Level System Architecture

The Warehouse Management System (WMS) is designed using a three-tier web application architecture composed of a Frontend Presentation Layer, a Backend Application Layer, and a Database Persistence Layer. This architecture ensures modularity, scalability, and maintainability across the entire platform.

The system is accessed through a web browser and is structured around role-based access control (RBAC), which separates the system capabilities for three user types: Master, PIC (Person-In-Charge), and QC.

Figure 1a — Client & Frontend Layer

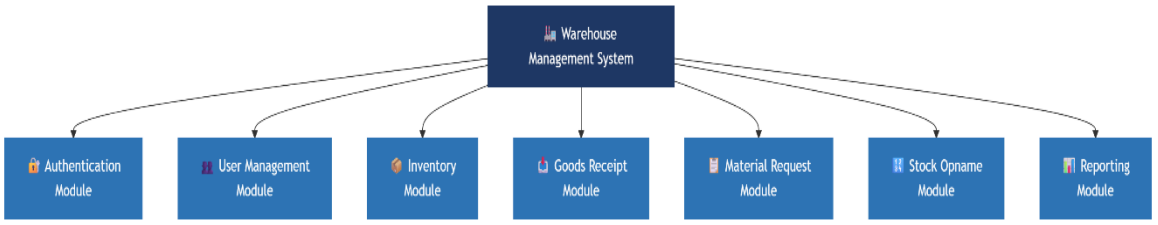


Figure 1a: Client & Frontend Layer (React.js + Tailwind CSS)

Figure 1b — Backend & Business Logic Layer

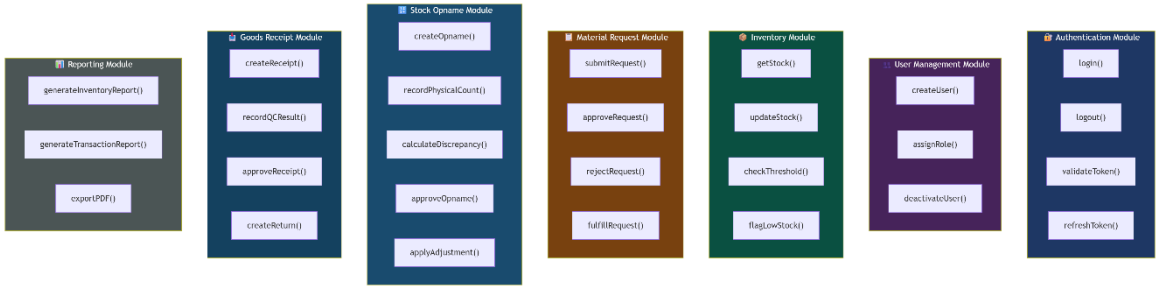


Figure 1b: Backend Layer — Laravel 10 REST API, Controllers, Services & Engines

Figure 1c — Database & External Services Layer

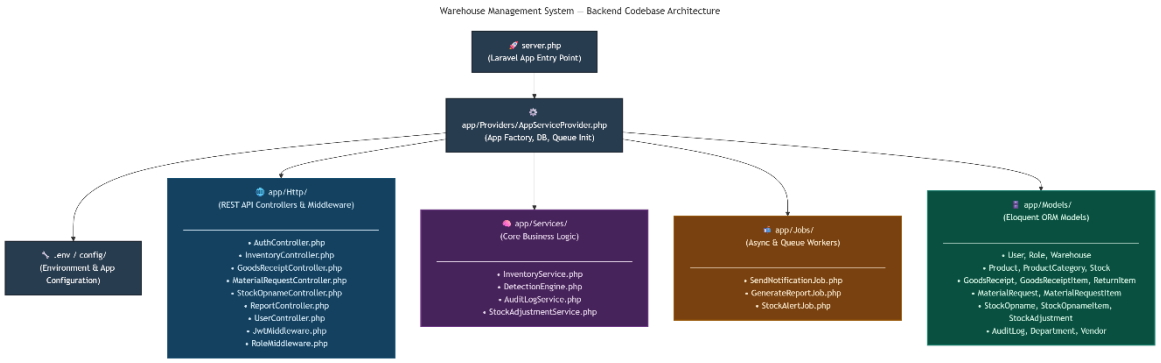


Figure 1c: Database Layer — MySQL 8.0 Table Groups & External SMTP Service

The architecture is composed of the following layers:

- Frontend Layer: Built using React.js, this layer renders the user interface for all roles. It communicates with the backend exclusively through RESTful API calls.
- Backend Layer: Built with Laravel (PHP), this layer handles business logic, authentication (JWT-based), and routing. It exposes a REST API consumed by the frontend.
- Database Layer: A MySQL relational database stores all persistent data including users, inventory, transactions, and audit logs.
- Third-party Integrations: Email notifications are handled via SMTP. Report exports use DomPDF library.

Within the system, data integrity is enforced at several architectural layers. When a user submits a transaction (e.g., goods receipt or material request), the system enforces multi-level validation:

- Input Validation: The frontend validates form fields before submission; the backend re-validates all incoming data server-side.
- Role Authorization Check: Every API endpoint checks the requesting user's role via JWT middleware before executing any business logic.
- Data Consistency Enforcement: All stock mutation operations are wrapped in database transactions to ensure ACID compliance.
- Audit Trail: Every state-changing operation records the responsible user, timestamp, and before/after values in dedicated log tables.

b. Asynchronous Processing & Integrations

Certain operations in the WMS are designed to be asynchronous to prevent blocking the main application thread:

- Email Notification Queue: When a material request is submitted or a goods receipt is completed, notification emails are queued using Laravel's job queue system and dispatched asynchronously.
- Stock Recalculation: After a stock opname is approved, the stock adjustment propagation is handled as a background job.
- Report Generation: For large data sets, report generation is queued and the user is notified upon completion.

c. Dashboard & User Interface

The system provides a centralized operational visibility dashboard. A dedicated dashboard accessible by the Master role provides a consolidated view of all active warehouse operations, including pending approvals, stock alerts, and recent transaction activity.

2. Technology Stack

The following table summarizes the technology stack used in the development of the Warehouse Management System:

No	Layer / Component	Technology	Description
1	Frontend Framework	React.js	SPA-based UI with component-driven architecture
2	CSS Framework	Tailwind CSS	Utility-first styling for responsive design
3	Backend Framework	Laravel 10 (PHP)	REST API, routing, middleware, job queuing
4	Authentication	JWT (JSON Web Token)	Stateless, role-based token authentication
5	Database	MySQL 8.0	Relational database for all persistent storage
6	ORM	Eloquent ORM	Database abstraction layer built into Laravel
7	API Architecture	RESTful API	Standard HTTP methods, JSON payload
8	Version Control	Git / GitHub	Source code management and team collaboration

No	Layer / Component	Technology	Description
9	Testing Tool	Postman	API endpoint testing and documentation
10	Deployment	Apache / Nginx	Web server for production hosting

3. Interface Mock-up Design

The following section presents the interface mock-up designs for all major screens of the Warehouse Management System.

a. Login Page

The Login Page serves as the entry point of the system. Users input their registered email and password. The system validates credentials against the database; upon success, a JWT token is issued and the user is redirected to their role-specific dashboard.

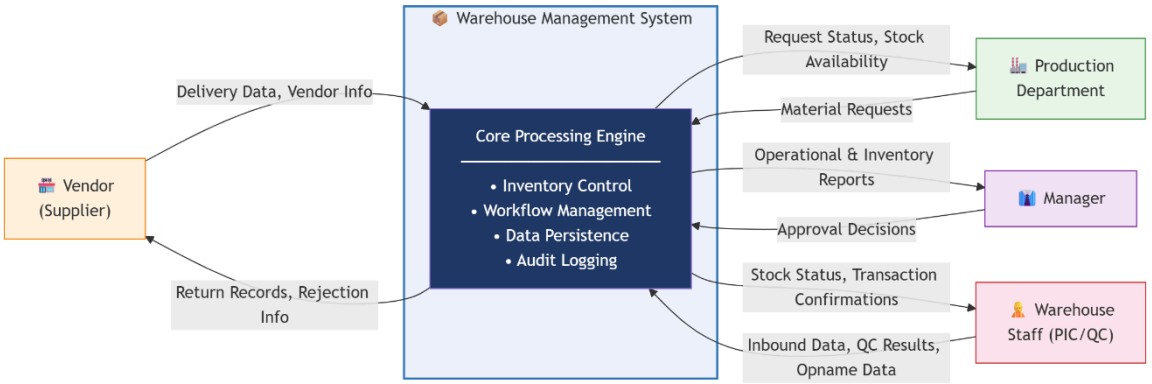


Figure 2a: Login Page — Authentication & Role Routing Flow

b. Master Dashboard

The Master Dashboard provides a consolidated operational view. It displays key performance indicators (KPIs) including total active stock, pending material requests, open goods receipts, and recent stock opname status.

c. Goods Receipt List

This page displays a paginated list of all goods receipts recorded in the system. Each entry shows the receipt ID, vendor name, receipt date, total items, QC status, and the PIC responsible.

d. Goods Receipt Detail – Before QC

This screen shows the detailed view of a goods receipt that has not yet undergone Quality Control inspection. The PIC can view the expected items and initiate the QC process.

e. Goods Receipt Detail – Post QC

After the QC inspection is completed, this screen displays the final reconciliation view. Accepted items are listed with their updated stock quantities. Rejected items appear with their reason for rejection and the return record reference.

f. Vendor Return Records

This page manages vendor return records. When goods are rejected during QC and returned to a vendor, a return record is created and listed here. The Master can review return frequency per vendor.

g. Stock Alert Rules

This configuration page allows the Master to define stock threshold rules for each product. When inventory falls below the minimum threshold, the system flags the product as low stock.

h. Create Stock Alert Rule

This form view allows the Master to create a new stock threshold rule. The Master selects a product, sets the minimum quantity threshold, and optionally specifies a notification recipient.

i. Real-Time Inventory Monitor

This screen provides a real-time view of current inventory levels across the warehouse. Products are displayed in a tabular format with their current stock quantity, minimum threshold, last transaction type, and last updated timestamp.

j. Settings (User & System Settings)

The Settings page allows administrators to manage user accounts, roles, and system configurations. The Master can create new user accounts, assign or change roles, reset passwords, and deactivate accounts.

B. HIERARCHICAL/ITERATIVE DESIGN

1. Module Hierarchy and Package Structure

The Warehouse Management System is decomposed into functional modules that form a hierarchical structure. Each module at a higher level orchestrates the operations of lower-level modules.

Figure 3a — Module Hierarchy (Top Level)

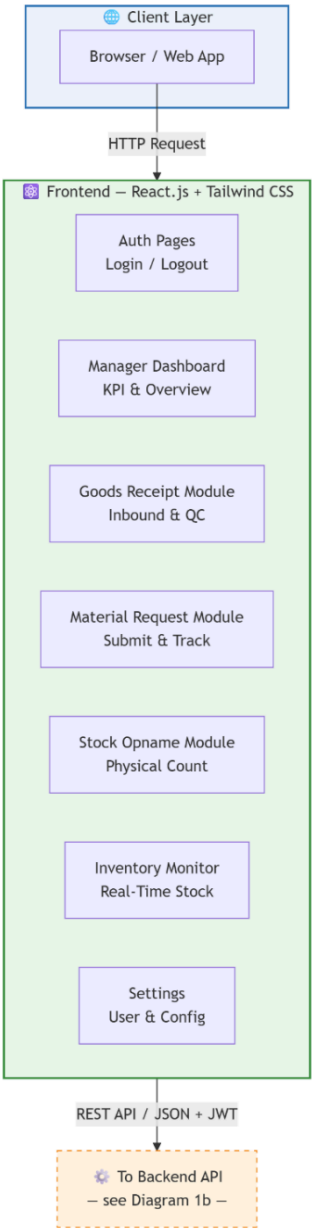


Figure 3a: Top-level Module Hierarchy — 7 Major Modules of the WMS

Figure 3b — Primitive Functions per Module

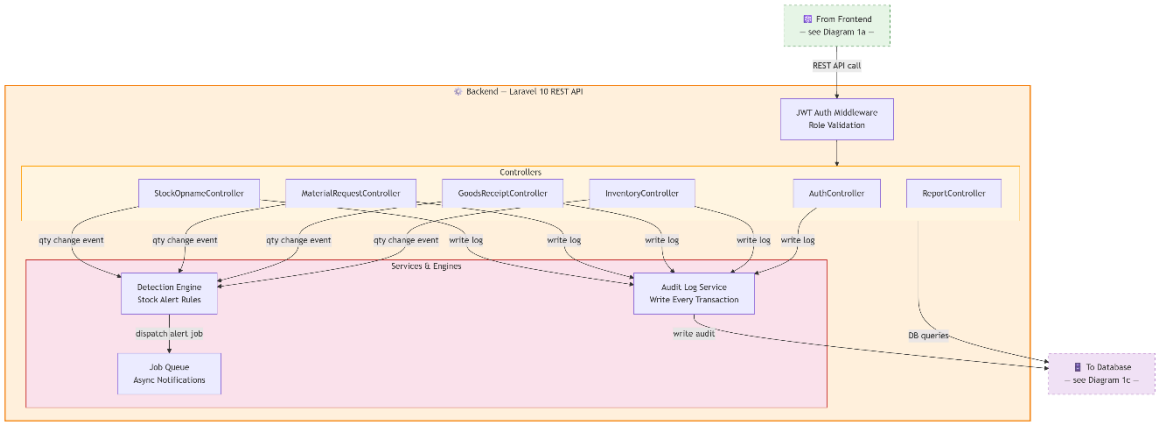


Figure 3b: Primitive Software Functions organized by Module

Figure 3c — Backend Codebase Architecture

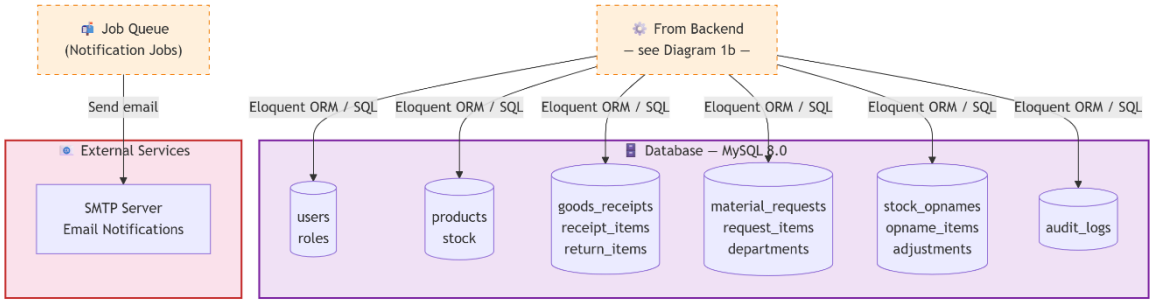


Figure 3c: Backend Codebase File Structure — Laravel Package Organization

The top-level modules and their sub-modules are organized as follows:

- Authentication Module: login(), logout(), validateToken(), refreshToken()
- User Management Module: createUser(), updateUser(), assignRole(), deactivateUser()
- Inventory Module: getStock(), updateStock(), checkThreshold(), flagLowStock()
- Goods Receipt Module: createReceipt(), recordQCResult(), approveReceipt(), createReturn()
- Material Request Module: submitRequest(), approveRequest(), rejectRequest(), fulfillRequest()
- Stock Opname Module: createOpname(), recordPhysicalCount(), calculateDiscrepancy(), approveOpname(), applyAdjustment()
- Reporting Module: generateInventoryReport(), generateTransactionReport(), exportPDF()

2. Block Diagram — Level 0 (Highest Level)

At Level 0, the system is represented as a single processing unit — the Warehouse Management System — that interacts with four external entities: Vendor, Production Department, Master, and PIC/QC Staf

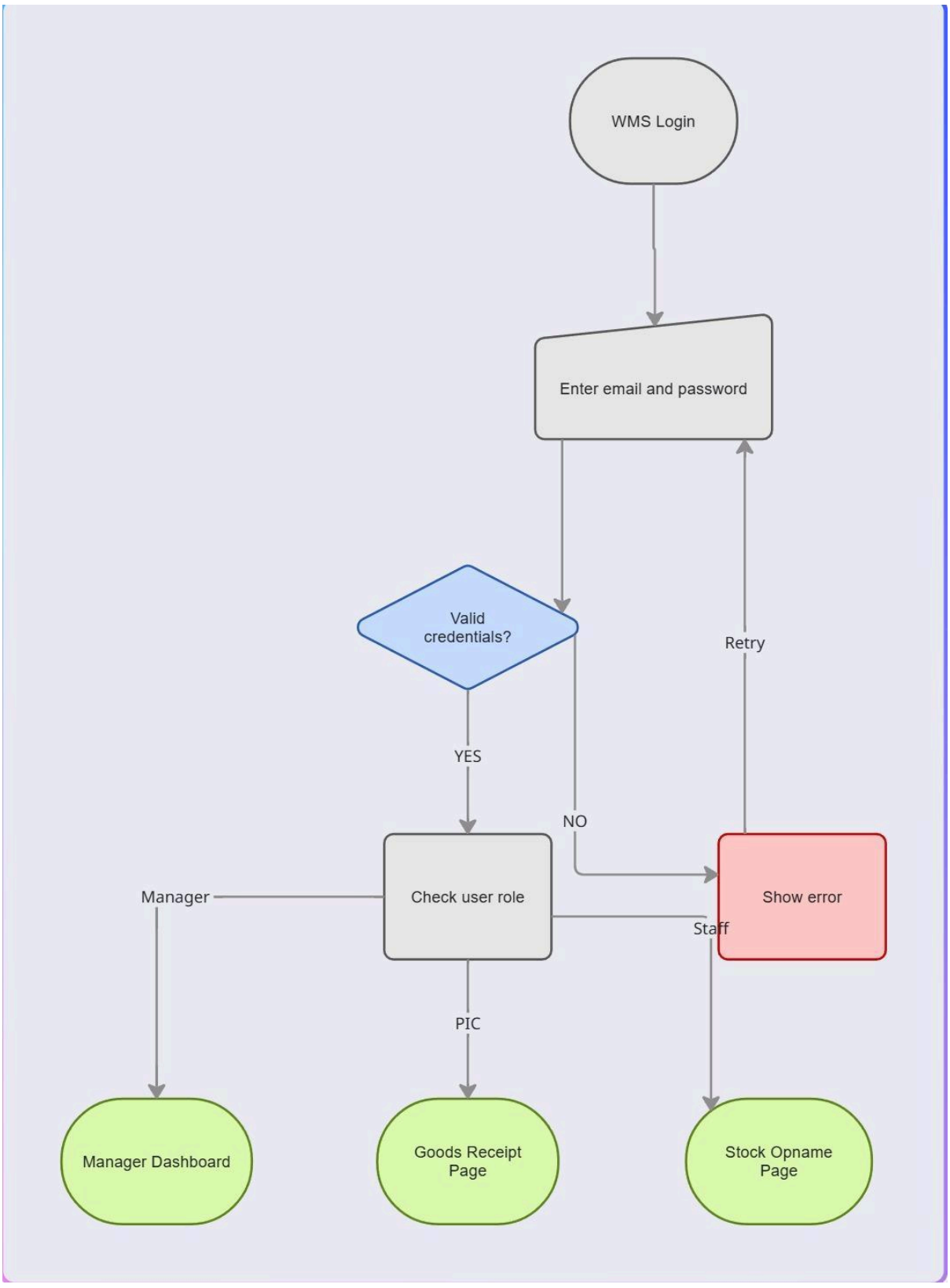


Figure 4: Block Diagram Level 0 — Context Diagram showing all external entity interactions

This Level 0 diagram corresponds to the Context Diagram described in Form 2, Section 2.5A. The system receives delivery data from Vendors; material requests from the Production Department; approval decisions from the Master; and inbound/outbound/opname data from PIC/QC Staff.

3. Block Diagram — Level 1 (Subsystem Level)

At Level 1, the WMS is broken down into its major subsystems. Each subsystem handles a distinct operational domain and exchanges data with other subsystems through defined interfaces.

Figure 5a — Subsystem Data Flow

*

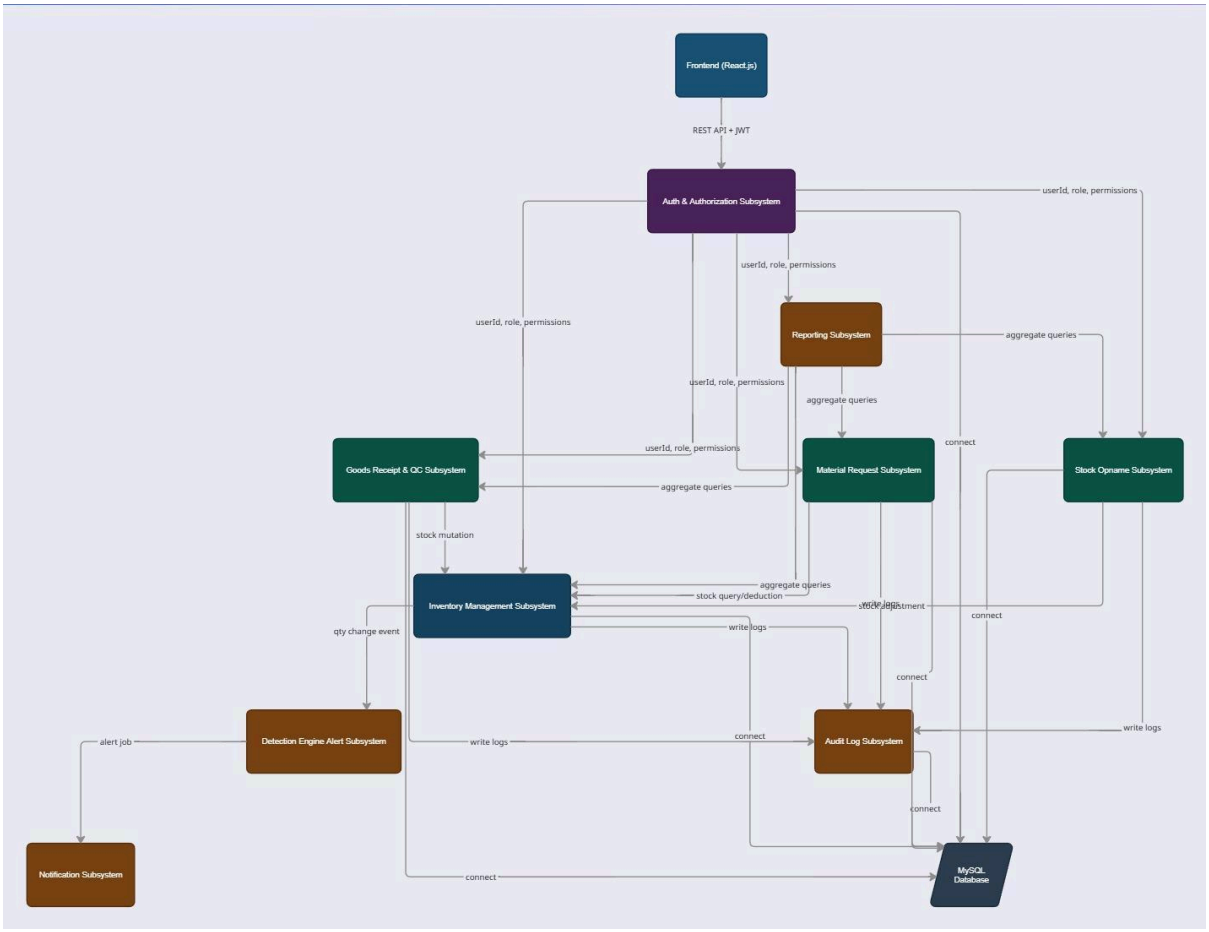


Figure 5a: Level 1 Subsystem Data Flow — showing inter-subsystem communication

Figure 5b — Subsystem to Database Table Mapping

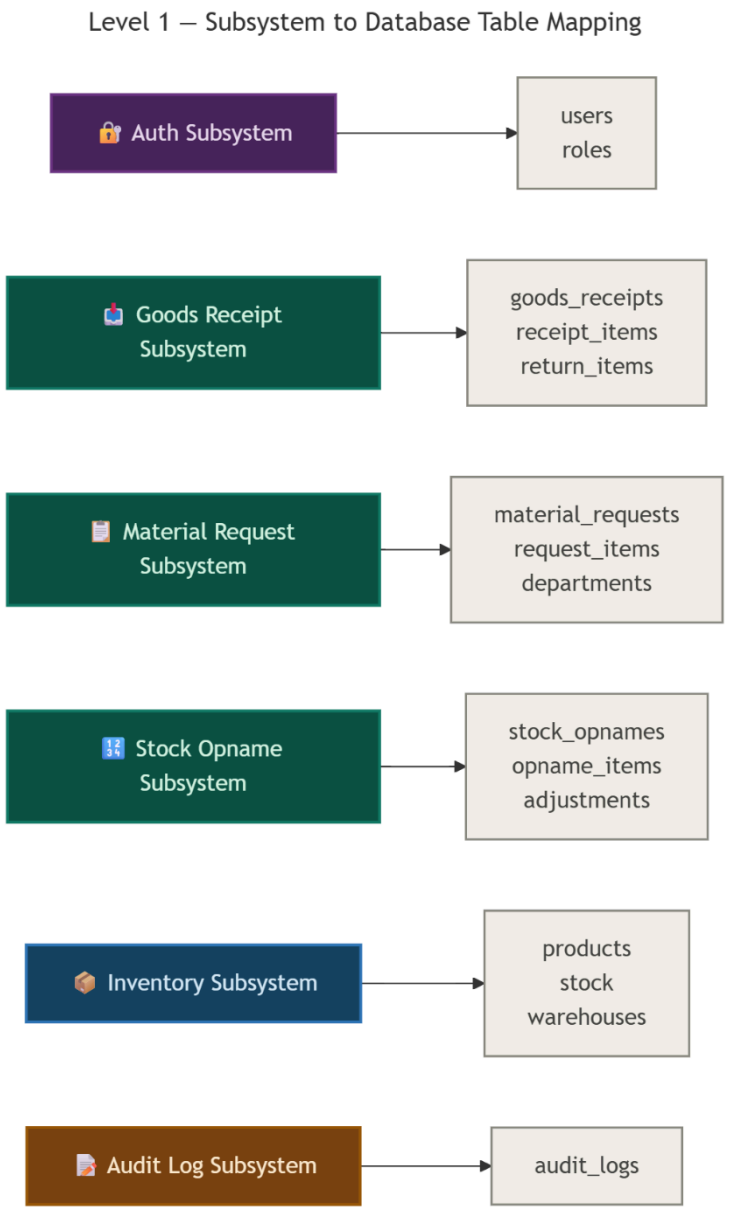


Figure 5b: Level 1 Subsystem to Database Table Mapping

The major subsystems are:

- Authentication & Authorization Subsystem: Handles user login, token management, and role-based access enforcement.
- Goods Receipt & QC Subsystem: Manages the receiving of goods from vendors, QC inspection workflow, and return processing.
- Material Request Subsystem: Handles request submission, approval workflow, stock availability check, and picking confirmation.
- Inventory Management Subsystem: Maintains real-time stock levels, enforces threshold rules, and serves stock queries.

- Stock Opname Subsystem: Manages periodic physical counting, discrepancy calculation, and adjustment approval.
- Reporting Subsystem: Aggregates data across subsystems to generate operational and inventory reports.

4. Block Diagram — Level 2 (Primitive Software Functions)

At Level 2, each subsystem is further decomposed into its primitive software functions. These are the lowest-level operations executed by the system.

a. Log Monitoring Module (Audit & Transaction Log)

Every state-changing operation in the system passes through the audit log module. The primitive functions include:

- writeLog(userId, action, tableName, recordId, oldValue, newValue, timestamp): Writes a single audit entry.
- queryLog(filters): Returns filtered log entries for display or reporting.
- archiveOldLogs(cutoffDate): Moves older log entries to an archive table to maintain performance.

b. Detection Engine (Stock Threshold & Alert Engine)

The detection engine evaluates stock levels against predefined rules after every inventory mutation. Its primitive functions include:

- loadThresholdRules(productId): Retrieves the threshold rule for a given product.
- evaluateThreshold(currentQty, minQty): Returns a boolean indicating whether the threshold is breached.
- triggerAlert(productId, warehouseId, currentQty): Creates an alert record and dispatches a notification job.
- resolveAlert(alertId): Marks an alert as resolved when stock is replenished.

c. Response Manager (Approval & Fulfillment Handler)

The response manager handles all approval-gated workflows. Primitive functions include:

- getPendingRequests(requestType): Queries pending records from the relevant table.
- approveRecord(recordId, approverId, timestamp): Updates the record status to 'approved'.
- rejectRecord(recordId, approverId, reason): Updates the record status to 'rejected' with a reason.

- notifyRequester(requesterId, result): Dispatches a notification to the requester.
- triggerPostApprovalAction(recordType, recordId): Calls the appropriate downstream action.

5. Interface Between Components

Components in the system communicate through well-defined interfaces. The following table summarizes the interface mechanisms between major subsystems:

No	From Component	To Component	Interface Type	Data Exchanged
1	Frontend (React)	Backend (Laravel)	REST API (HTTP/JSON)	JWT token, form payloads, query params
2	Auth Subsystem	All Subsystems	Middleware / Parameter Passing	userId, roleId, permissions object
3	Goods Receipt	Inventory Module	Function Call / DB Transaction	productId, qty, warehouseId, direction
4	Material Request	Inventory Module	Function Call / DB Transaction	productId, requestedQty, warehouseId
5	Stock Opname	Inventory Module	Function Call / DB Transaction	productId, adjustedQty, opnameId
6	Inventory Module	Detection Engine	Function Call	productId, warehouseId, newQty
7	Detection Engine	Notification Queue	Message Passing (Job Queue)	alertId, recipientId, message payload
8	Any Subsystem	Audit Log Module	Function Call	userId, action, tableName, old/new values
9	Backend	Database (MySQL)	PDO / Eloquent ORM	SQL queries, result sets

6. UML Diagrams

a. Class Diagram

The class diagram depicts the main entities (models) in the system and their relationships, corresponding directly to the database schema defined in Form 2.

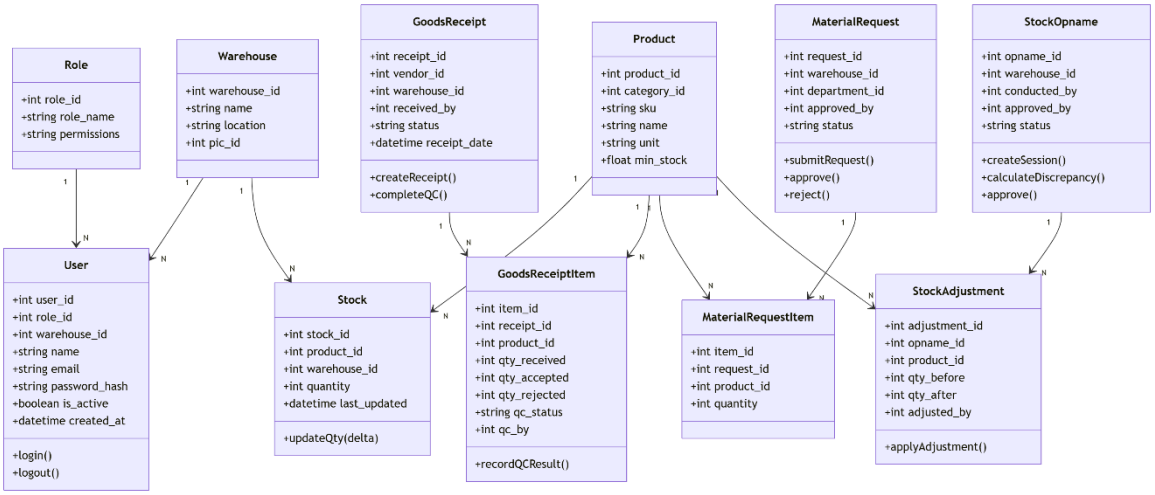


Figure 8: UML Class Diagram — Entity relationships and methods

b. Activity Diagram

The activity diagrams for all major business flows are inherited from Form 2 and extended in Form 3 to reflect the system's internal processing steps. The key flows are: (1) User Login & Role Management, (2) Goods Receipt & QC, (3) Material Request, (4) Stock Opname, and (5) Stock In/Out. Refer to Form 2, Section 2.5 for the base activity diagrams.

c. Sequence Diagram

The sequence diagram illustrates the message exchange between system components for the Material Request approval workflow — one of the most complex interaction flows in the system. The participants are: PIC/Dept User, Frontend (React), Backend (Laravel), AuthMiddleware, MaterialRequestController, InventoryService, NotificationService, and MySQL Database.

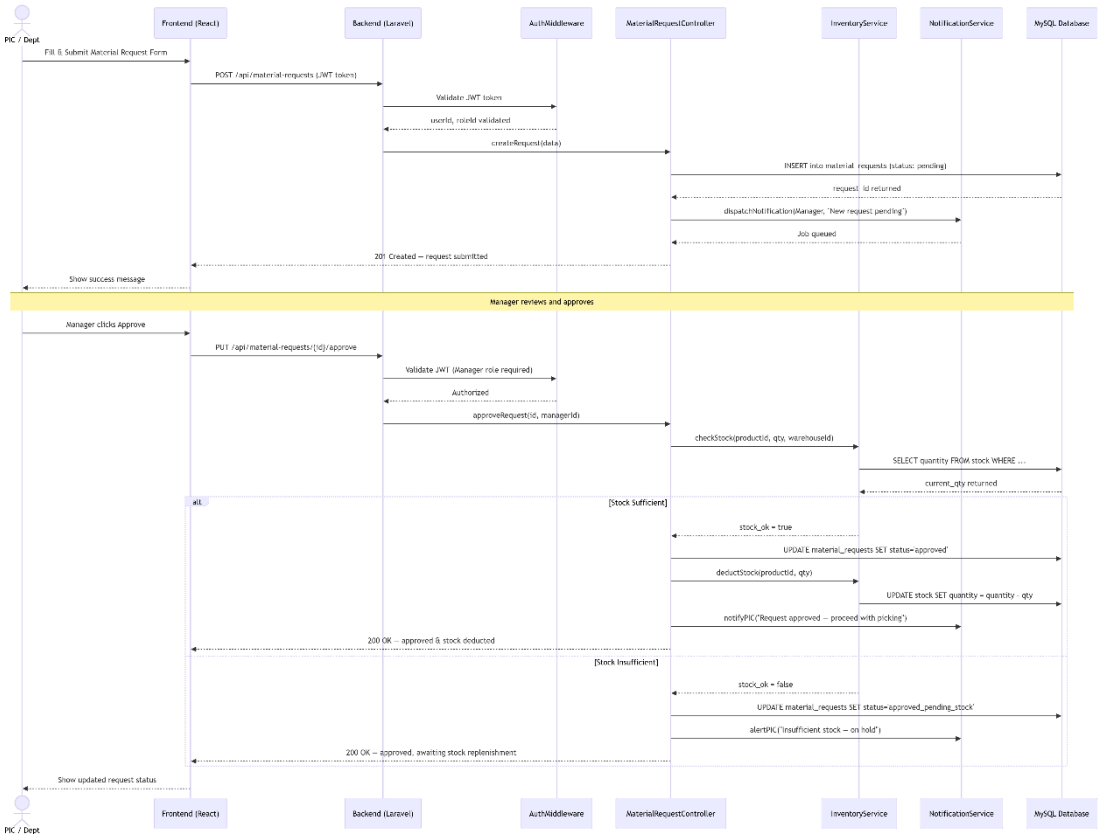


Figure 10: Sequence Diagram — Material Request Submission & Approval Flow

7. Component References and Libraries

The following table lists all major component references, libraries, and packages used in the development of the Warehouse Management System:

No	Library / Package	Version	Layer	Purpose
1	React.js	18.x	Frontend	UI component framework
2	React Router DOM	6.x	Frontend	Client-side routing and navigation
3	Axios	1.x	Frontend	HTTP client for REST API calls
4	Tailwind CSS	3.x	Frontend	Utility-first CSS styling framework
5	React Hook Form	7.x	Frontend	Form state management and validation
6	Laravel	10.x	Backend	PHP web framework for API and business logic
7	JWT Auth	3.x	Backend	API token authentication (JWT)
8	Eloquent ORM	Built-in	Backend	Database abstraction and query building

No	Library / Package	Version	Layer	Purpose
9	Laravel Queue	Built-in	Backend	Async job processing for notifications
10	MySQL	8.0	Database	Relational database management system
11	Composer	2.x	Backend	PHP dependency manager
12	npm	9.x	Frontend	Node.js package manager for frontend deps
13	Postman	Latest	Testing	API testing and documentation
14	Git / GitHub	Latest	DevOps	Version control and team collaboration
15	DomPDF (Laravel)	2.x	Backend	PDF report generation

C. STANDARDS USED

The following table lists all technical standards and specifications adopted in the design and implementation of the Warehouse Management System:

No	Standard / Specification	Category	Description & Application
1	REST (REpresentational State Transfer)	API Architecture	All client-server communication follows REST principles. HTTP verbs (GET, POST, PUT, DELETE) are used semantically.
2	JSON (RFC 8259)	Data Format	All API request and response payloads are encoded in JSON format. Consistent key naming uses snake_case.
3	JWT (RFC 7519)	Authentication	JSON Web Tokens are used for stateless authentication. Tokens include role and user ID claims and have a configurable expiry time.
4	HTTP/1.1 (RFC 7231)	Communication Protocol	Standard HTTP/1.1 protocol is used for all API communication between frontend and backend.
5	ISO 8601	Date & Time Format	All date and datetime values stored and transmitted use ISO 8601 format (YYYY-MM-DD, YYYY-MM-DDTHH:MM:SS).
6	UML 2.5	Modeling Standard	UML version 2.5 notation is used for all class diagrams, sequence diagrams, and activity diagrams in this document.
7	ACID (Database)	Data Integrity	All multi-step database operations are executed within ACID-compliant transactions in MySQL.
8	RBAC (Role-Based Access Control)	Authorization	User access is governed by predefined roles (Master, PIC, QC). Each role has a fixed permission set enforced at the middleware layer.
9	WCAG 2.1 (Level AA)	Accessibility	The UI is designed to meet Web Content Accessibility Guidelines Level AA, ensuring adequate color contrast and keyboard navigability.
10	PSR-12 (PHP)	Code Standard	PHP backend code follows PSR-12 coding style standards for consistency and readability.
11	ESLint (Airbnb Config)	Code Standard	Frontend JavaScript code is linted using ESLint with the Airbnb configuration profile.
12	UTF-8	Character Encoding	All text data stored in the database and transmitted via API uses UTF-8 encoding.

No	Standard / Specification	Category	Description & Application
13	HTTPS / TLS 1.2+	Security	All API communications in production are encrypted using TLS 1.2 or higher.

D. IMPLEMENTATION AND TESTING SCENARIO

1. Implementation Approach

The implementation of the Warehouse Management System follows an iterative and incremental development approach, aligned with the modules identified in the hierarchical design. Development is structured into five phases:

Phase	Module	Tasks	Implementation Notes
1	Foundation	Project setup, DB schema migration, authentication	Setup Laravel project, configure MySQL, implement JWT auth middleware, create all DB tables from Form 2 schema.
2	Inventory Core	Product, warehouse, stock management	Implement Eloquent models and REST endpoints for products, warehouses, and stock. Build inventory query APIs.
3	Operations	Goods receipt, QC, material request	Implement the goods receipt workflow including QC approval step and return processing. Build material request lifecycle with approval gate.
4	Stock Opname & Alerts	Opname, adjustments, threshold engine	Implement opname session management, discrepancy calculation, approval workflow, and automated stock adjustment. Integrate detection engine.
5	Frontend & Reporting	React UI, dashboards, report generation	Build React frontend components for all screens. Implement dashboard, inventory monitor, and report export features.

2. Functional Testing Scenarios

The following testing scenarios cover all major functional requirements. Each scenario includes the precondition, test steps, expected result, and the alternative/failure path.

Test Group 1: Authentication & Role Management

TC#	Test Case	Test Steps	Expected Result	Alternative / Failure Path
TC-01	Valid Login – Master	1. Open login page 2. Enter valid Master email & password 3. Click Login	JWT token issued. Redirected to Master Dashboard with full access.	TC-02: Invalid credentials — error message shown, no token issued.
TC-02	Invalid Login	1. Enter wrong password 2. Click Login	Error message: 'Invalid credentials'. No redirect.	After 5 failed attempts, account is temporarily locked for 15 minutes.
TC-03	PIC Role Redirect	1. Login as PIC user 2. Observe landing page	Redirected to Goods Receipt page. Master-only pages are inaccessible.	Attempting to access Master URLs returns 403 Forbidden response.
TC-04	QC Role Redirect	1. Login as QC user 2. Observe accessible pages	Only Goods Inspection page is accessible. Approval and report pages are restricted.	403 Forbidden on all other routes.
TC-05	Logout	1. Click Logout 2. Attempt to access protected page	JWT token invalidated. User redirected to login page. Protected page returns 401.	Expired token on protected route also triggers redirect to login.

Test Group 2: Goods Receipt & Quality Control

TC#	Test Case	Test Steps	Expected Result	Alternative / Failure Path
TC-06	Create Goods Receipt	1. PIC logs in 2. Navigate to Goods Receipt 3. Select vendor, fill item quantities 4. Submit	Goods receipt created with status 'pending_qc'. Stock not yet updated.	If vendor not in system: error 'Vendor not found'.
TC-07	QC – All Items Pass	1. Open pending receipt 2. Set all items as 'accepted' 3. Submit QC result	Receipt status 'completed'. Stock quantities increased for all accepted items. Audit log created.	If QC submitted by non-PIC role: 403 Forbidden.
TC-08	QC – Partial Reject	1. Open pending receipt 2. Accept some items, reject others 3. Submit	Accepted items update stock. Rejected items generate return records. Receipt marked 'partially_completed'.	If rejection reason is empty: validation error.
TC-09	QC – All Rejected	1. Reject all items in a receipt 2. Submit QC	Receipt status set to 'rejected'. All items recorded in return_items. Stock unchanged.	Stock values must remain unchanged after submission.

Test Group 3: Material Request Workflow

TC#	Test Case	Test Steps	Expected Result	Alternative / Failure Path
TC-10	Submit Material Request	1. PIC logs in 2. Navigate to Material Request 3. Select product, quantity, department 4. Submit	Request created with status 'pending'. Notification sent to Master.	If quantity is 0 or negative: validation error.
TC-11	Approve – Stock Available	1. Master opens pending request 2. Clicks Approve	Request approved. Stock decremented. PIC notified to proceed with picking.	If stock check fails mid-transaction: rollback, status remains 'pending'.
TC-12	Approve – Stock Insufficient	1. Master approves a request where qty > available stock	Request approved but flagged 'insufficient stock'. PIC notified. Fulfillment on hold.	Stock is not decremented. Alert raised.
TC-13	Reject Material Request	1. Master opens pending request 2. Clicks Reject 3. Enters reason	Request status 'rejected'. PIC and requester notified. Stock unchanged.	If reason field is empty: validation error.

Test Group 4: Stock Opname

TC#	Test Case	Test Steps	Expected Result	Alternative / Failure Path
TC-14	Create Opname Session	1. PIC logs in 2. Navigate to Stock Opname 3. Click 'New Opname' 4. Confirm warehouse	Opname session created with status 'in_progress'. All products listed with system_qty pre-filled.	If another opname is active for same warehouse: error 'An active opname already exists'.
TC-15	No Discrepancy	1. PIC enters physical counts matching system qty for all products 2. Submit	Discrepancy shows 0 for all items. Opname status 'completed'. No adjustment required.	If physical_qty is left blank: validation error.
TC-16	With Discrepancy	1. PIC enters physical counts different from system qty 2. Submit	Discrepancy calculated and shown. Opname status 'completed'. Sent to Master for approval.	System must correctly calculate: discrepancy = physical_qty - system_qty.
TC-17	Master Approves Opname	1. Master reviews opname with discrepancies 2. Clicks Approve	Stock adjustments applied. Adjustment records created. Stock table updated. Audit log written.	If adjustment fails for any item: full rollback.

3. Success Metrics

The following success metrics define the criteria for evaluating the correctness and completeness of the system implementation:

No	Metric	Target	Measurement Method
1	Authentication Success Rate	100% of valid credentials granted; 100% of invalid rejected	Manual test cases TC-01 to TC-05
2	Stock Accuracy after Goods Receipt	Stock qty matches sum of accepted items across all QC tests	Compare stock table values after TC-06 to TC-09
3	Material Request Fulfillment	Stock decremented correctly upon approval when stock sufficient	Verify stock before and after TC-11
4	Opname Adjustment Accuracy	Stock adjustments exactly match discrepancies calculated during opname	Verify stock table after TC-17 against opname_items records
5	Role-Based Access Control	No user can access pages or APIs outside their role's permission set	Attempt cross-role access in all TC groups
6	Audit Log Completeness	Every state-changing transaction creates an audit log entry	Query audit_logs table after each test group
7	Transaction Atomicity	No partial data writes; all multi-step operations fully succeed or rollback	Simulate failure mid-transaction and verify DB state

REFERENCES

1. Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.
2. Pressman, R. S., & Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill.
3. Laravel Documentation. (2024). Authentication, Eloquent ORM, Queues. https://laravel.com/docs
4. React Documentation. (2024). React 18 Official Docs. https://react.dev
5. MySQL Documentation. (2024). MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/
6. IETF. (2015). RFC 7519 – JSON Web Token (JWT). https://tools.ietf.org/html/rfc7519
7. Object Management Group. (2017). UML 2.5.1 Specification. https://www.omg.org/spec/UML/
8. W3C. (2018). Web Content Accessibility Guidelines (WCAG) 2.1. https://www.w3.org/TR/WCAG21/